

Magic Methods

Python reserves method names with the unusual form of two underscores, some name, and two underscores for a particular purpose:

```
__<someName>__()
```

These are officially called *special methods* but are more commonly referred to by Python programmers as *magic methods*. Many of these are already defined, such as `__init__()`, which is called whenever you instantiate an object from a class, but all other names in this style are available for future expansion. These are known as “magic” methods because Python calls them behind the scenes whenever it detects an operator, a special function call, or some other special circumstance. They are not intended to be called by client code directly.

NOTE

Because the names of these magic methods are difficult to pronounce—for example, `__init__()` is read as “underscore underscore init underscore underscore”—Python programmers often refer to these as dunder methods (a shortened version of double underscore). This method would be referred to as “dunder init.”

Continuing with the previous examples, we’ll look at how this works with the `+` operator. The built-in data types (integer, float, string, Boolean, and so on) are actually implemented as classes in Python. We can see this by testing with the built-in `isinstance()` function, which takes an object and a class and returns `True` if the object was instantiated from the class or `False` if not. These lines will both report `True`:

```
print(isinstance(123, int))
print(isinstance('some string', str))
```

The classes for the built-in data types contain a set of magic methods, including ones for the basic math operators. When Python detects the `+` operator with integers, it calls the magic method named `__add__()` in the built-in integer class, which performs integer addition. When Python sees